

LyZR Manager

LESSON 02 · DESIGN & CREATE

What you'll learn

- Build a team of specialist agents and put a manager in charge, on the Orchestrate canvas
- Turn an ordinary agent into a manager by connecting it to its sub-agents — no routing code
- Hand the manager a request with two jobs buried in it and watch it route to the right specialists
- Read the activity panel to see which sub-agents were called and how their answers were merged

Key concepts

A single agent is great right up until one question has two jobs buried in it. "My order isn't here, and I want to return something" is really two requests. Rather than build one giant agent that tries to do everything, you build a team of specialists and put a manager in charge of routing between them.

A manager is born from a connection, not a special agent type. On the Orchestrate canvas you drop in your agents, then connect one lead agent to the others — that connection is what promotes it to manager. Watch the badge change: the support agent becomes the manager, and order-lookup and returns become its sub-agents. Each specialist stays good at exactly one thing.

The manager decides the order of operations itself — you never script it. Give it a request that needs two specialists and it works out the sequence on its own: it checks its own knowledge, calls order-lookup, calls returns, then merges everything into one coherent answer. You compose the team; the manager handles the choreography.

To the user it's still one conversation. The customer asked one messy question and got one clean reply — order 5832 has shipped, expected Tuesday via UPS, with an offer to escalate, plus the full return policy and a prompt for the item's condition. The complexity lives behind the scenes, not in the user's experience.

Reach for a manager when the work is open-ended and you can't script the steps in advance. If you could lay the steps out as a fixed sequence, you'd want a SuperFlow instead. The manager earns its place precisely when the path can't be known up front.

In Studio

Studio feature: **Orchestrate** → **Manager** (the agent-composition canvas).

TAKEAWAY

A single agent is great right up until one question has two jobs buried in it.

TAKEAWAY

A manager is born from a connection, not a special agent type.

TAKEAWAY

The manager decides the order of operations itself — you never script it.

TAKEAWAY

To the user it's still one conversation.

TAKEAWAY

Reach for a manager when the work is open-ended and you can't script the steps in advance.

1. From the home screen, open **Orchestrate** and go into the **Manager** canvas — this is where you compose a team of agents.
2. **Search your agents** and bring in the team: a **support agent** that knows your policies, an **order-lookup agent**, and a **returns agent**.
3. Drop the **support agent** on first, then add the other two. Each is a single-focus specialist.
4. **Connect** the support agent to the other two. That connection promotes support to **manager**; order-lookup and returns become its **sub-agents** (watch the badge).
5. **Open the manager and chat** with something that needs both: *"My order 5832 still isn't here, and I want to return one of the items."*
6. Open the **activity panel** to watch what it actually did — checks its own knowledge, calls order-lookup, calls returns, then composes one reply.
7. Confirm the merged answer covers both jobs: order status (with an escalation offer) and the return policy (with a prompt for the item and its condition).

TRY THIS

- 1** On the Orchestrate canvas, build a three-agent team for a domain you know — e.g. a front-desk agent plus two specialists — and connect the front-desk agent to the other two to make it the manager.
- 2** Send the manager a deliberately *two-part* question and open the activity panel. Trace which sub-agents it called and in what order — confirm you never had to specify that order.
- 3** Now ask a question only *one* specialist can answer. Watch the manager call just that agent. *Notice it doesn't over-delegate* — it routes to exactly what the request needs.